

Reviewer Pack — Cyclotomic Norm Floor of Truth-Table Character Sum Caps $\text{ACC}^\wedge \dots$

Entry a13755e34e77 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 20:35:08 UTC

Cyclotomic Norm Floor of Truth-Table Character Sum Caps $\text{ACC}^\wedge 0[m]$ Size

Entry ID: a13755e34e77 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 20:35:08 UTC

1. Conjecture

Field A (mathematical branch): Algebraic number theory of cyclotomic integers — the field norm $N_{\{Q(\zeta_m)/Q\}: Z[\zeta_m] \rightarrow Z}$ computed as the determinant of the multiplication-by- α regular representation on the basis $\{1, \zeta_m, \dots, \zeta_m^{\varphi(m)-1}\}$ reduced modulo Φ_m , in the Stickelberger-Lehmer-Mahler tradition (rarely deployed in circuit complexity; distinct from Mahler-measure-of-truth-table polynomial which is blacklisted, since the input here is a sum of roots of unity $\zeta_m^{\text{int}(x) \bmod m}$ living natively in O_K rather than a root-of-an-integer-polynomial height). **Field B** (complexity object): $\text{ACC}^\wedge 0[m]$ circuit size $s_{\{\text{ACC}[m]\}}(f)$ of Boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, equivalently the Beigel-Tarui $\text{SYM} \cdot \text{AND}$ size, the canonical Williams-2011 algorithmic-method target for $\text{NEXP} \not\subseteq \text{ACC}^\wedge 0$.

Statement:

Fix $m \geq 2$ and let $\zeta_m = e^{2\pi i/m}$. For $f: \{0,1\}^n \rightarrow \{0,1\}$ define $\alpha_f = (1/2^n) \cdot \sum_{x \in \{0,1\}^n} f(x) \cdot \zeta_m^{\text{int}(x) \bmod m} \in Q(\zeta_m)$. Then there exist absolute constants C, m_0 such that for every $m \geq m_0$ with at least two distinct prime factors and every f computable by an $\text{ACC}^\wedge 0[m]$ circuit of size $s \geq 2$, the cyclotomic norm satisfies $-\log_2 |N_{\{Q(\zeta_m)/Q\}}(\alpha_f)| \leq C \cdot \varphi(m) \cdot \log_2 s$; equivalently, $|N(\alpha_f)| \geq s^{\{-C \cdot \varphi(m)\}}$. A single (f, s) pair with $|N(\alpha_f)| < s^{\{-100 \cdot \varphi(m)\}}$ refutes the conjecture.

Rationale (proposer's reasoning):

By Beigel-Tarui every $\text{ACC}^\wedge 0[m]$ function is a SYM of s AND s, so α_f decomposes additively into s elementary character sums each with constant-norm contributions in $O_K = Z[\zeta_m]$; sub-multiplicativity of the absolute norm under sums (via the Hadamard-style determinant bound on the regular representation) suggests the total norm cannot dip below $s^{\{-O(\varphi(m))\}}$. Random f have $|\alpha_f| \approx 2^{\{-n/2\}}$, hence $|N(\alpha_f)| \approx 2^{\{-n\varphi(m)/2\}}$, exponentially below the bound — exposing exactly the algebraic gap Williams' algorithmic method exploits between structured ACC computations and generic truth tables. The cyclotomic ring is the natural home for MOD_m structure, yet has not been used as a circuit-size obstruction.

Taxonomy category: $\text{ACC_LB_via_WILLIAMS}$ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 10362861991d6ec5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $m \in \{6, 10, 15\}$ and $n \in \{8, 10, 12, 14\}$, run ≥ 400 ACC⁰[m] SYM•AND circuits per (m,n) with $s \in \{4, 8, 16, 32, 64\}$ (≥ 3 seeds aggregated). Regress $y = -\log_2 |N(\alpha_f)|$ on $x = \log_2 s$. SUPPORT iff in $\geq 10/12$ (m,n) cells: OLS slope $\leq 3 \cdot \phi(m)$ AND 95th-percentile residual $\leq \phi(m)$, while $\geq 90\%$ of 200 random-TT and MAJ_n controls give $y > 3 \cdot \phi(m) \cdot n$. FALSIFY iff any ACC sample yields $y > 100 \cdot \phi(m) \cdot \log_2 s$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	UNCERTAIN	0.85	HITS	SAFE
ALGEBRIZATION	SAFE	0.62	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - cyclotomic field norm ACC⁰ circuit lower bound roots of unity - Stickelberger Boolean function character sum circuit complexity - ACC[m] correlation roots of unity Z[zeta_m] Beigel Tarui symmetric

Top relevant hits considered: - [<http://arxiv.org/abs/1611.06783v2>] Cyclotomic polynomials at roots of unity - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/2110.13086v2>] Quantum Algorithms and Lower Bounds for Linear Regression with Norm Constraints - [<http://arxiv.org/abs/2603.27466v1>] English translation of Frobenius' and Stickelberger's "On the theory of elliptic functions" - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/1303.2001v9>] Representations of quantum groups at roots of unity, Whittaker vectors and q-W algebras - [<http://arxiv.org/abs/2006.06610v3>] Classification of Hopf superalgebras associated with quantum special linear superalgebra at roots of unity using Weyl gr

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from sys import argv
```

```

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def prime_factors(n):
    factors = []
    d = 2
    while n >= d * d:
        if n % d:
            d += 1
        else:
            n //= d
            factors.append(d)
    if n > 1:
        factors.append(n)
    return factors

def phi(n):
    result = n
    p = 2
    while p * p <= n:
        if n % p == 0:
            while n % p == 0:
                n //= p
            result -= result // p
        p += 1
    if n > 1:
        result -= result // n
    return result

def generate_truth_table(n):
    return [random.randint(0, 1) for _ in range(2 ** n)]

def generate_acc_circuit(m, s, n):
    circuit = []
    for _ in range(s):
        gate = random.choice(['AND', 'NOT'])
        if gate == 'NOT':
            inputs = [random.randint(0, 1)]
        else:
            inputs = [random.randint(0, 1) for _ in range(random.randint(2, int(math.log2(n)) + 1))]
        circuit.append((gate, inputs))
    return circuit

```

```

def evaluate_circuit(circuit, input_vector):
    stack = []
    for gate, inputs in reversed(circuit):
        if gate == 'NOT':
            stack.append(not input_vector[inputs[0]])
        else:
            result = input_vector[inputs[0]]
            for i in range(1, len(inputs)):
                if gate == 'AND':
                    result &= input_vector[inputs[i]]
                elif gate == 'OR':
                    result |= input_vector[inputs[i]]
            stack.append(result)
    return stack.pop()

def compute_alpha_f(truth_table, m):
    n = int(math.log2(len(truth_table)))
    alpha_f = 0
    for x in range(2 ** n):
        k = sum(x & (1 << i) for i in range(n))
        alpha_f += truth_table[x] * (complex(math.cos(2 * math.pi * k / m), math.sin(2 * math.pi * k / m)))
    return alpha_f / (2 ** n)

def compute_cyclotomic_norm(alpha_f, m):
    n = int(math.log2(len(truth_table)))
    reg_rep_matrix = []
    for x in range(m):
        row = [complex(math.cos(2 * math.pi * i * x / m), math.sin(2 * math.pi * i * x / m)) for i in range(n)]
        reg_rep_matrix.append(row)
    det = 1
    for i in range(n):
        det *= reg_rep_matrix[i][i]
    return abs(det)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    m_values = [6, 10, 15]
    n_values = [8, 10, 12, 14]
    s_values = [4, 8, 16, 32, 64]
    num_trials = 400
    controls = 200

    results = []
    for m in m_values:
        if len(prime_factors(m)) < 2:
            continue
        for n in n_values:
            for s in s_values:
                for _ in range(num_trials):
                    circuit = generate_acc_circuit(m, s, n)
                    truth_table = generate_truth_table(n)
                    alpha_f = compute_alpha_f(truth_table, m)
                    norm = compute_cyclotomic_norm(alpha_f, m)
                    results.append(-math.log2(norm), math.log2(s))

    for _ in range(controls):

```

```

truth_table = generate_truth_table(random.randint(8, 14))
alpha_f = compute_alpha_f(truth_table, random.choice([6, 10, 15]))
norm = compute_cyclotomic_norm(alpha_f, random.choice([6, 10, 15]))
results.append(-math.log2(norm), math.log2(2 ** random.randint(8, 14)))

for _ in range(controls):
    n = random.randint(8, 14)
    alpha_f = compute_alpha_f([random.randint(0, 1) for _ in range(2 ** n)], random.choice([6, 10, 15]))
    norm = compute_cyclotomic_norm(alpha_f, random.choice([6, 10, 15]))
    results.append(-math.log2(norm), math.log2(2 ** n))

slope, intercept = 0, 0
for y, x in results:
    slope += (y - intercept) * x / len(results)
    intercept += (y - intercept) * (1 - x) / len(results)

mean_slope = sum(y - intercept for y, _ in results) / len(results)
std_slope = math.sqrt(sum((y - intercept - mean_slope) ** 2 for y, _ in results) / len(results))
mean_resid = sum(abs(y - (intercept + slope * x)) for y, x in results) / len(results)
std_resid = math.sqrt(sum((abs(y - (intercept + slope * x)) - mean_resid) ** 2 for y, x in results) / len(results))

conjecture_holds = all(slope <= 3 * phi(m) and mean_resid <= phi(m) for m in m_values for n in n_values)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "slope",
    "metric_value": slope,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in argv[1:]] if argv[1:] else [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_slope = sum(r["metric_value"] for r in results) / len(results)
    std_slope = math.sqrt(sum((r["metric_value"] - mean_slope) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_fraction}")
    elif any(r["metric_value"] > 100 * phi(m) * math.log2(s) for m, n, s in [(6, 8, 4), (6, 8, 8)]):
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={seeds[0]}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d2ef101.py", line 166, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d2ef101.py", line 126, in run_trial
    norm = compute_cyclotomic_norm(alpha_f, m)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d2ef101.py", line 98, in compute_cyclotomic_norm
    n = int(math.log2(len(truth_table)))
                        ^^^^^^^^^^^^^^^
```

NameError: name 'truth_table' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('truth_table' is not defined) inside compute_cyclotomic_norm before any data was produced, so neither the support nor falsification conditions can be evaluated. | next: Fix the variable scoping bug in compute_cyclotomic_norm (pass the truth table explicitly) and rerun the pre-registered sweep over $m \in \{6, 10, 15\}$ and $n \in \{8, 10, 12, 14\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	174914
2	preregistration	claude_max	opus	0	0	7960
3	novelty	claude_max	opus	0	0	6123
4	novelty	claude_max	opus	0	0	8006
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19097
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20775
7	judge	claude_max	opus	0	0	4662

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 241537 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/a13755e34e77.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a13755e34e77.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a13755e34e77.tar.gz` (if generated)